```
# Data handling libraries
import pandas as pd # Used for data manipulation and analysis, especially with
import numpy as np # Used for numerical operations, especially with arrays

# Visualization libraries
import matplotlib.pyplot as plt # Used for creating static, interactive, and ar
import seaborn as sns # Built on Matplotlib, provides a high-level interface fc

# Dataset library from Hugging Face
from datasets import load_dataset # Used to easily load and share datasets from

# Transformer libraries
from transformers import BertTokenizer, BertForSequenceClassification, Trainer,

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, cc
```

```
# Load SMS spam dataset from Hugging Face Datasets Hub
dataset = load_dataset("sms_spam")

# Convert the 'train' split of the dataset to a pandas DataFrame for easier ins
df = dataset["train"].to_pandas()

# Display the first 5 rows of the DataFrame to inspect the data structure and c
print(df.head())
```

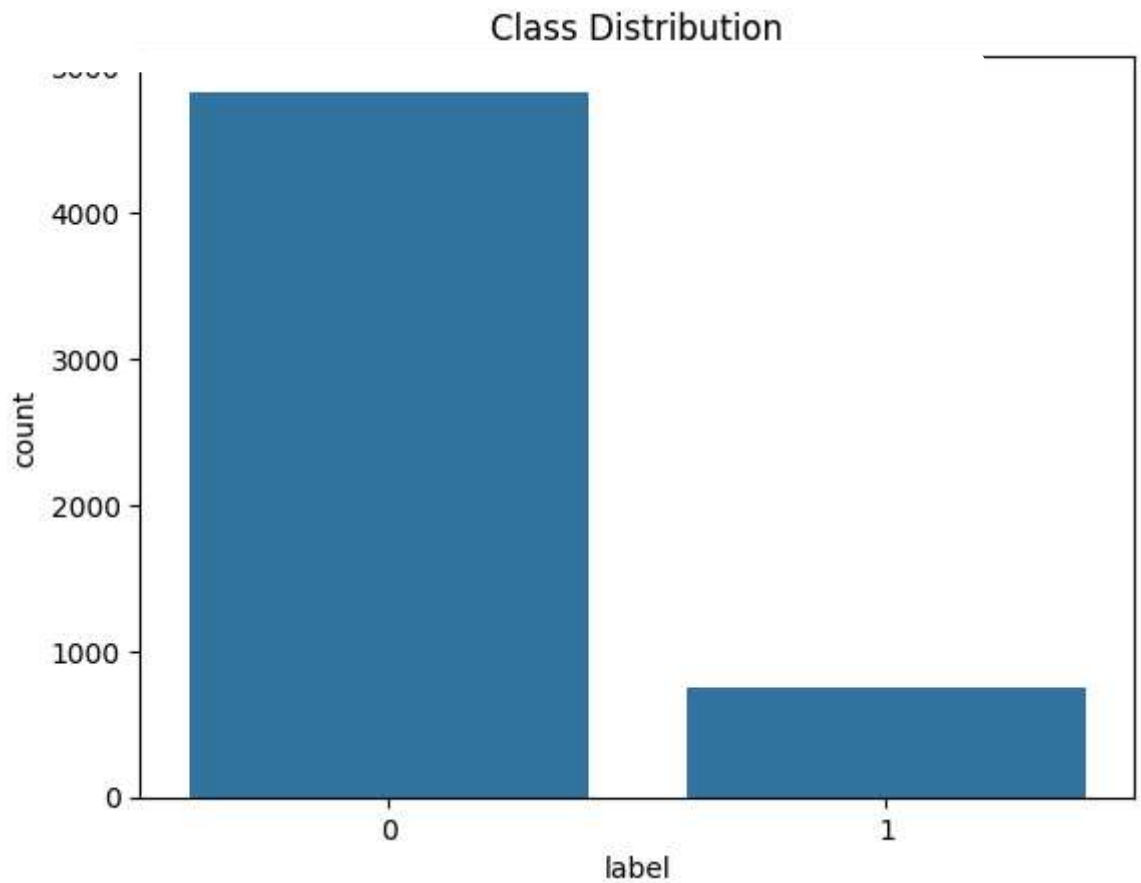
```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserW
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
                                                 sms  label
0  Go until jurong point, crazy.. Available only ...      0
1                     Ok lar... Joking wif u oni...\n      0
2  Free entry in 2 a wkly comp to win FA Cup fina...      1
3  U dun say so early hor... U c already then say...      0
4  Nah I don't think he goes to usf, he lives aro...      0
```

```
# Print the names of all columns in the DataFrame to understand the available f
print(df.columns)
```

```
Index(['sms', 'label'], dtype='object')
```

```
# Create a countplot to visualize the distribution of the 'label' column (0 for
sns.countplot(x=df['label'])
```

```
# Set the title of the plot for clarity
plt.title("Class Distribution")
# Display the plot
plt.show()
```




```
# Initialize the BERT tokenizer for the 'bert-base-uncased' model.
# This tokenizer converts text into numerical tokens that the BERT model can ur
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
def tokenize(example):
    # Tokenize the 'sms' text, adding padding, truncation, and setting a max_le
    # This prepares the text data into a format suitable for BERT.
    return tokenizer(example["sms"], padding="max_length", truncation=True, max

# Apply the 'tokenize' function to the entire dataset using the .map() method.
# 'batched=True' processes multiple examples at once for efficiency.
tokenized_dataset = dataset.map(tokenize, batched=True)
```

Map: 100% 5574/5574 [00:01<00:00, 3075.65 examples/s]

```
# Rename the 'label' column to 'labels' to match the expected input name for Hu
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")
# Set the format of the dataset to 'torch' tensors and specify the columns to b
```

```
# This prepares the dataset for use with PyTorch models and the Trainer.
tokenized_dataset.set_format("torch", columns=["input_ids", "attention_mask", "
```

```
# Split the 'train' portion of the tokenized dataset into training and testing
# 'test_size=0.2' means 20% of the data will be used for testing, and 80% for t
split = tokenized_dataset["train"].train_test_split(test_size=0.2)

# Assign the training split to 'train_dataset'
train_dataset = split["train"]
# Assign the testing split to 'test_dataset'
test_dataset = split["test"]
```

```
# Load a pre-trained BERT model for sequence classification.
# 'bert-base-uncased' specifies the base BERT model without a cased vocabulary.
# 'num_labels=2' indicates that the model will classify into 2 categories (e.g.
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_
```

```
Loading weights: 100%   199/199 [00:00<00:00, 539.58it/s, Materializing param=bert.pooler.dense.we

BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
classifier.bias                             | MISSING    |
classifier.weight                           | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; r
```

```
# Update the transformers library to its latest version.
# This ensures access to the newest features, bug fixes, and API changes.
!pip install -U transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/pyt
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/di
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/pyt
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/d
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dis
```

```
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pack
```

```
# Defines training parameters for model training process
training_args = TrainingArguments(

    output_dir="./results",          # Directory where training outputs will be

    learning_rate=2e-5,              # Controls how fast the model learns

    per_device_train_batch_size=8,   # Number of samples processed at once duri

    per_device_eval_batch_size=8,    # Batch size used during evaluation

    num_train_epochs=3               # Number of times the model sees entire da
)
```

```
# This function calculates model performance metrics
# It is used by Trainer during evaluation phase
def compute_metrics(pred):

    # Extracts actual labels from predictions
    labels = pred.label_ids

    # Converts model output probabilities into predicted class labels
    preds = pred.predictions.argmax(-1)

    # Calculates precision, recall and F1 score
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds,

    # Calculates overall accuracy of predictions
    acc = accuracy_score(labels, preds)

    # Returns all evaluation metrics as dictionary
    return {
        "accuracy": acc,
        "precision": precision,
```

```
            "recall": recall,
            "f1": f1,
        }
```

```
# Creates Trainer object to handle training and evaluation automatically
trainer = Trainer(

    model=model,                     # The BERT classification model

    args=training_args,              # Training configuration parameters

    train_dataset=train_dataset,     # Dataset used for training

    eval_dataset=test_dataset,       # Dataset used for evaluation

    compute_metrics=compute_metrics  # Function to calculate evaluation metrics
)
```

```
# Starts training process of the Transformer model
# It updates model weights based on training data
trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: User
  super().__init__(loader)
```

[237/1674 45:38 < 4:39:03, 0.09 it/s, Epoch 0.42/3]

Step	Training Loss